

Time Complexity

Friday, May 1, 2026

19:46

Time Complexity — Easy Notes

Time complexity sounds scary, but it is actually very simple.

It does **not** mean actual clock time (like 2 seconds or 5 seconds).

It means:

How fast or slow your code grows when input becomes bigger

That's it.

Time complexity helps us measure **efficiency**.

1. What Time Complexity Really Means

Imagine you have a small list:

[1, 2, 3]

Now imagine a huge list:

[1, 2, 3, 4, 5, 6, ..., 100000]

A code that works fast for 3 items may become very slow for 100,000 items.

So time complexity tells us:

“If input gets bigger, how much more work will my code do?”

2. Why It Matters

Two solutions can give the same answer.

But one may be:

- fast
- efficient
- scalable

And the other may be:

- slow
- wasteful
- bad for large input

Time complexity helps us choose the better solution.

3. Big-O Notation

We write time complexity using **Big-O**.

Big-O means:

“How does runtime grow as input size grows?”

We usually write:

- $O(1)$
- $O(n)$
- $O(n^2)$
- ...

- $O(\log n)$

These are just ways to describe speed.

4. $O(1)$ — Constant Time

This means the work stays the same no matter how big input is.

Example:

```
arr = [10, 20, 30, 40]
```

```
print(arr[2])
```

Accessing one index is always one step.

Whether array has:

- 4 items
- 400 items
- 400000 items

Still one direct lookup.

So this is $O(1)$.

Easy meaning:

Same work every time

Fastest type.

5. $O(n)$ — Linear Time

This means work grows with input size.

Example:

```
for num in arr:
```

```
    print(num)
```

If array has:

- 5 items → 5 steps
- 100 items → 100 steps
- 100000 items → 100000 steps

Work grows linearly.

So this is $O(n)$.

Easy meaning:

More input = more work

Very common.

6. $O(n^2)$ — Quadratic Time

This usually happens with nested loops.

Example:

```
for i in arr:
```

```
    for j in arr:
```

```
        print(i, j)
```

If $n = 5$, work = 25

If $n = 100$, work = 10000

That grows very fast.

So this is $O(n^2)$.

Easy meaning:

Loop inside loop = much slower
This becomes expensive quickly.

7. $O(\log n)$ — Logarithmic Time

This is very efficient.

It happens when we reduce the problem size again and again.

Example: Binary Search

Instead of checking every element,
we cut the search space in half each time.

Like searching in dictionary:

- open middle
- go left or right
- cut half again

That is $O(\log n)$.

Easy meaning:

Keep cutting problem into half
Very fast for big inputs.

8. Real-Life Analogy

Imagine finding a name in a phone book.

$O(n)$

Start from first page and check one by one.

Slow.

$O(\log n)$

Open in middle, then half again.

Much faster.

That is the difference.

9. Common Time Complexities (Easy Ranking)

From fastest to slowest:

- $O(1)$ → best
- $O(\log n)$ → very fast
- $O(n)$ → good
- $O(n \log n)$ → okay
- $O(n^2)$ → slow
- $O(2^n)$ → very slow
- $O(n!)$ → terrible

As complexity grows, performance gets worse.

10. Easy Way to Recognize Them

One direct operation

One direct operation

Usually $O(1)$

Example:

```
arr[0]
```

One loop

Usually $O(n)$

Example:

```
for x in arr:
```

Nested loops

Usually $O(n^2)$

Example:

```
for i in arr:
    for j in arr:
```

Halving each time

Usually $O(\log n)$

Example:

Binary Search

11. Why We Ignore Small Details

In time complexity, we ignore small constants.

Example:

```
for i in arr:
```

```
    print(i)
```

```
for i in arr:
```

```
    print(i)
```

This is technically $2n$, but in Big-O we write:

$O(n)$

Because Big-O cares about growth, not tiny details.

Same idea:

- $3n + 5 \rightarrow O(n)$
- $n^2 + n \rightarrow O(n^2)$

We only keep the dominant term.

12. Main Goal of Time Complexity

The goal is simple:

Write code that still works fast when input becomes huge.

That is why time complexity matters.

Not just “does it work?”

But:

“Does it work efficiently?”

That is what good coding means.

Final Easy Summary

- Time complexity tells how code performance grows
- It measures efficiency
- Big-O is used to describe it
- $O(1)$ = constant
- $O(n)$ = one loop
- $O(n^2)$ = nested loops
- $O(\log n)$ = divide in half
- Lower complexity = faster code
- Higher complexity = slower code

Easy memory trick:

- One operation $\rightarrow O(1)$
- One loop $\rightarrow O(n)$
- Loop inside loop $\rightarrow O(n^2)$
- Cut in half $\rightarrow O(\log n)$

That's the core idea.

===== TRICKS & GOLDEN RULES =====

Time Complexity Tricks (The Easy Way I'd Remember It)

This is the simplest way to stop getting confused.

Do **not** memorize definitions first.

Learn to **spot patterns**.

That is the trick.

Trick 1: Count Loops First

The fastest way to guess time complexity:

Look at loops.

This solves most beginner confusion.

Rule:

- No loop $\rightarrow O(1)$
- One loop $\rightarrow O(n)$
- Loop inside loop $\rightarrow O(n^2)$
- 3 nested loops $\rightarrow O(n^3)$

Example:

`x = arr[0]`

No loop → **$O(1)$**

for x in arr:

 print(x)

One loop → **$O(n)$**

for i in arr:

 for j in arr:

 print(i, j)

Two loops → **$O(n^2)$**

This is the easiest first trick.

Trick 2: Ask “How Many Times Will This Run?”

This one question solves most problems:

 “How many times is this code running?”

That is time complexity.

Example:

for i in range(n):

Runs n times → **$O(n)$**

for i in range(n):

 for j in range(n):

Runs $n * n$ → **$O(n^2)$**

Always ask:

 how many total executions?

That’s the answer.

Trick 3: Ignore Constants

This is the biggest beginner shortcut.

Big-O does **not** care about small numbers.

Example:

for i in range(n):

 print(i)

for i in range(n):

 print(i)

This runs $2n$ times.

But we write:

$O(n)$

Why?

Because Big-O only cares about growth.

So:

- $2n \rightarrow O(n)$
- $5n \rightarrow O(n)$
- $100n \rightarrow O(n)$

Ignore constants.

Easy trick:

 Remove numbers, keep growth.

Trick 4: Keep the Biggest Term Only

If multiple terms exist, keep only the biggest one.

Example:

- $n^2 + n \rightarrow O(n^2)$
- $n^2 + 100 \rightarrow O(n^2)$
- $n + \log n \rightarrow O(n)$

Why?

Because for very large input, biggest term dominates.

Easy trick:

Keep the slowest-growing monster.

Trick 5: Nested Loop Is Not Always $O(n^2)$

This is an important trap.

Beginners see nested loop and panic.

But nested loop is **not always** $O(n^2)$.

Example:

```
i = 0
```

```
j = 0
```

```
while i < n and j < n:
```

```
    i += 1
```

```
    j += 1
```

Looks like two variables.

But total work is still just $O(n)$.

Why?

Because both move forward together.

So trick is:

Don't count loops. Count total movement.

Very important.

Trick 6: Two Pointers Often Means $O(n)$

This is a huge interview trick.

Example:

```
left = 0
```

```
right = n - 1
```

```
while left < right:
```

```
    left += 1
```

```
    right -= 1
```

Looks like two moving parts.

But each pointer moves only across array once.

So total work = $O(n)$

Easy trick:

Two pointers usually still mean one pass.

Trick 7: Halving Means $O(\log n)$

If problem keeps shrinking by half:

It is usually **$O(\log n)$**

Words to watch for:

- half
- divide by 2
- binary search
- repeatedly reduce

Easy trick:

If size keeps getting cut in half $\rightarrow O(\log n)$

Trick 8: Recursion Shortcut

Quick recursion trick:

- One recursive call $\rightarrow$ usually **$O(n)$**
- Two recursive calls $\rightarrow$ often **$O(2^n)$**

Example:

```
def f(n):  
    return f(n-1)
```

One call each time $\rightarrow$ **$O(n)$**

```
def f(n):  
    return f(n-1) + f(n-2)
```

Two calls branching $\rightarrow$ much slower

Usually **$O(2^n)$**

Easy trick:

Branching recursion grows fast

Trick 9: Best Question to Ask in Interviews

When stuck, ask yourself:

1. Is there a loop?

$\rightarrow$ count it

2. Is there a nested loop?

$\rightarrow$ multiply work

3. Is input shrinking?

$\rightarrow$ think log

4. Are pointers only moving forward?

$\rightarrow$ probably linear

These 4 questions solve most beginner problems.

Trick 10: Beginner Cheat Sheet

Memorize this:

- One lookup $\rightarrow$ **$O(1)$**
- One loop $\rightarrow$ **$O(n)$**
- Two nested loops $\rightarrow$ **$O(n^2)$**

- Cut in half → $O(\log n)$
- Two pointers → often $O(n)$
- Recursion with branching → often expensive

This alone solves a lot.

Easiest Way to Think About Time Complexity

Do not ask:

“What is the formula?”

Ask:

“How much work grows when input grows?”

That is the easiest way to think.

Golden Shortcut

If confused, use this order:

1. Count loops
2. Count total work
3. Ignore constants
4. Keep biggest term
5. Watch for halving

That is the easiest beginner method.

===== ADDITIONAL NOTES =====

1. Time Complexity is About Growth, Not Exact Time

This is the most important mindset.

Time complexity does **not** mean:

- 2 seconds
- 5 milliseconds
- actual stopwatch time

It means:

“How does work grow when input grows?”

That’s it.

A program can take 1 second on one computer and 3 seconds on another.

But its time complexity stays the same.

So Big-O measures **growth**, not actual clock speed.

This is one of the biggest beginner misunderstandings.

2. Input Size = n

You will keep seeing n everywhere.

n simply means:

size of input

Example:

arr = [10, 20, 30, 40]

Here, n = 4

If array has 100 items, then n = 100

So whenever you see:

- **$O(n)$**
- **$O(n^2)$**
- **$O(\log n)$**

That n just means input size.

Very simple, but very important.

3. Best Case, Worst Case, Average Case

This is often skipped early, but useful.

A code may not always take the same time.

Example: searching for 5 in array

[5, 1, 2, 3, 4]

Found immediately → very fast

But:

[1, 2, 3, 4, 5]

Need to check all items

So one algorithm can behave differently depending on input.

That gives:

- **Best case** → fastest
- **Worst case** → slowest
- **Average case** → normal expected

In interviews, we usually discuss **worst case** unless told otherwise.

Easy trick:

If not mentioned, think worst case.

4. Space Complexity

Very important and often forgotten.

Time complexity asks:

How long does it take?

Space complexity asks:

How much extra memory does it use?

Example:

arr = [1, 2, 3]

This already uses memory.

But if your solution creates another large list, dictionary, or stack, memory usage increases.

That is **space complexity**.

So every solution has 2 costs:

- Time
- Space